E-commerce Listing + Cart + Filters (URL synced)

- **Problem:** Build a product listing page like Flipkart / Amazon / Myntra. Show products in a grid with search, category checkboxes, a price range, a minimum rating, an "in stock only" toggle, sorting and pagination. Every filter lives in the URL, so a link can be shared, a refresh keeps the view, and the browser back button undoes the last filter. Users add products to a cart, change quantities (never above stock) and see the subtotal. The cart survives a refresh.
- **Tech stack:** React (Vite, JavaScript, Tailwind CSS v4)
- **Time limit:** 60 minutes
- **Often asked at:** Flipkart, Amazon, Myntra, Meesho
- **Why it matters:** Filters synced to the URL, cart state

PART A: Thought process

General steps for any machine coding task

1. **Understand the requirements.** Restate the problem, split functional vs non-functional, ask clarifying questions, write down assumptions.
2. **List the interactions.** Every user action and its expected result. This is your checklist for building and testing.
3. **Design the architecture.** Name the components and their one-line jobs. Start small.
4. **Design the state.** What is the single source of truth? What can be derived?
5. **Design data structures and contracts.** Shape of the data, API inputs and outputs, URL format.
6. **Trace one data flow end to end.** Proves the design works before you type.
7. **Find the hard parts.** Solve them on paper first.
8. **Build in layers.** Simplest working version first, then one working layer at a time.
9. **Cover states and edge cases.** Loading, error, empty, invalid input, stale data.
10. **Wrap up.** Trade-offs and what you would add with more time.

Time budget for 60 minutes

Phase	Time
Requirements + clarifying questions	5 min
Interactions, architecture, state design	7 min
Hard parts on paper (URL as state, debounced search, cart reconcile)	5 min
Build: mock API + product grid with loading / error	8 min
Build: pure filter / sort / paginate functions	8 min
Build: URL hook (parse, serialize, push, popstate) + filter panel	12 min

Phase	Time
Build: cart reducer + persistence + drawer	10 min
Edge cases, accessibility, wrap-up	5 min

Tip: think out loud

Say what you are doing and why, even when it feels obvious: "I'll keep filters in the URL and not in useState, because then back, refresh and share all work for free." The interviewer grades your reasoning as much as the code. If you are stuck, say what you are trying; silence looks like being lost.

PART B: Apply it to this problem

1. Understand the requirements

Functional requirements

- Load products from an API and show them in a responsive grid (image, title, category, rating, price, stock state).
- Search by text (debounced).
- Filter by category (multi-select, with a count next to each option), price range (min / max), minimum rating, in stock only.
- Sort: relevance, price low to high, price high to low, top rated, newest.
- Paginate: 12 per page.
- All of the above lives in the URL: shareable link, refresh-safe, back / forward works.
- Cart: add, increase, decrease, remove, clear. Quantity never goes above stock. Badge with item count, drawer with lines and subtotal.
- Cart survives refresh (localStorage).

Non-functional requirements

Area	What it means here
Performance	Filtering is a pure function wrapped in `useMemo`; search is debounced; one card update re-renders one card (`memo`).
Scalability	Filter logic takes the same filter object the server would take, so moving to server-side filtering changes only the fetch.
Error handling	Fetch failure shows a retry; bad URL params fall back to defaults; broken localStorage gives an empty cart, never a crash.
Accessibility	Labels on every input, `fieldset` / `legend` for groups, `aria-live` result count, `aria-current` on the page button, cart is a dialog with focus management and Escape.
Usability	Clean URLs (defaults left out), "Clear all", empty state with a reset button, out-of-stock items can't be added.

Area	What it means here
Reusability / extensibility	Adding a filter = one field in `DEFAULT_FILTERS`, one line each in parse, serialize and match.

Clarifying questions

1. Is filtering done on the client (all products in one response) or on the server (API takes the filters)?
2. Which filters must be in the URL? All of them, including sort and page?
3. Should every filter change create a history entry, or should typing in search replace the current entry?
4. Is the cart only on this device (localStorage), or synced to a user account?
5. Does the stock number come from the listing API, and can it change while the cart is open?
6. Pagination or infinite scroll?

Assumptions we will build with

- The API returns the full catalog (48 products); we filter, sort and paginate on the client.
- Every filter, the sort and the page are in the query string. Defaults are left out of the URL.
- Checkbox / select / page changes **push** a history entry; debounced search typing **replaces** it.
- Cart is guest-only, stored in localStorage as `{ productId: qty }`. Prices always come from the latest catalog.
- Quantity is capped at the product's stock. Checkout and payment are out of scope.
- Numbered pagination, 12 per page.

2. Identify interactions / use cases

Action / input	Expected behavior / output
Open the page	Skeleton grid, then products. URL params (if any) are applied.
API fails	Error message with a Retry button.
Type in search	Input updates instantly; after 300 ms of no typing the URL gets `?q=` (replace) and results update.
Press Enter in search	Commit immediately, no wait.
Tick a category	URL gets `cat=...` (push), results and counts update, page resets to 1.
Set price min / max and press Go	URL gets `min` / `max`. If min > max they are swapped.
Pick rating / tick in stock	URL updates, results update, page resets to 1.
Change sort	URL gets `sort=`, order changes, page resets to 1.
Click page 3	URL gets `page=3`, grid shows items 25-36, scroll to top.
Click browser Back	Previous filters come back, inputs show them (including the search box).
Open a shared link	Same results as the sender. Invalid params are ignored.
Old link with `page=99`	Shows the last page instead of an empty grid.
Filters match nothing	"No products match" with a clear button.

Action / input	Expected behavior / output
Click Add to cart	Button turns into a − qty + stepper; badge count goes up.
Click + at stock limit	+ is disabled; reducer also ignores it.
Click − at qty 1 / Remove	Line is removed from the cart.
Out of stock product	"Out of stock" text, disabled Add button.
Open cart	Drawer with lines, line totals, subtotal; focus moves to Close.
Escape / backdrop / ×	Drawer closes, focus goes back to the cart button.
Refresh	Filters (URL) and cart (localStorage) both survive.
Saved cart qty > current stock	Quantity clamped, note "Only N left, quantity updated".
Cart changed in another tab	This tab picks up the new cart (`storage` event).

3. Design the architecture

Module	Responsibility
`api/productsApi.js`	Mock catalog and `fetchProducts()` with latency and random failure.
`utils/filters.js`	Pure: `parseFilters` , `serializeFilters` , `applyFilters` , `getCategoryCounts` , `paginate` .
`utils/cart.js`	Pure: `cartReducer` , `loadCart` (sanitise storage), `getCartSummary` (join with catalog).
`hooks/useUrlFilters.js`	Reads filters from `location.search` , writes with push / replace, listens to back / forward.
`hooks/useProducts.js`	Fetch status machine: loading / success / error + retry.
`hooks/useCart.js`	`useReducer` + persist to localStorage + cross-tab sync.
`App.jsx`	Wires hooks together, derives the visible list, renders states.
`SearchBox`	Local draft + debounced commit to the URL.
`FilterPanel` / `PriceFilter`	Category checkboxes with counts, price form, rating, in stock, clear all.
`ProductCard` / `QtyStepper`	One product; add button or quantity stepper.
`Pagination`	Prev / numbers / Next.
`CartDrawer`	Dialog with cart lines and subtotal.

```
App
├─ useUrlFilters()  ─ URL  (q, cat, min, max, rating, stock, sort, page)
├─ useProducts()    ─ API  (catalog)
├─ useCart()        ─ localStorage ({ id: qty })
```

```
│
├── header
│   ├── SearchBox
│   └── Cart button (badge)
├── FilterPanel
│   └── PriceFilter
├── results
│   ├── sort select + count
│   ├── ProductCard × 12 ── QtyStepper
│   └── Pagination
└── CartDrawer ── QtyStepper per line
```

Start with everything in `App` plus the pure utils. Split out `SearchBox`, `PriceFilter` and `CartDrawer` when they get their own local state or effects.

4. Design the state / data model

State or field	Purpose
URL `location.search`	**Source of truth for filters, sort and page.** Not copied into React state.
`products` + `status` + `error`	Catalog and fetch status (`useProducts`).
`cart` = `{ [productId]: qty }`	What the user picked (`useCart` , persisted).
`cartOpen`	Is the drawer open (UI only).
`SearchBox.text`	Draft text while typing (local).
`PriceFilter.draftMin / draftMax`	Draft numbers before Go (local).

```
// Parsed from the URL on every change
const DEFAULT_FILTERS = {
  q: '', categories: [], min: null, max: null,
  rating: 0, inStock: false, sort: 'relevance', page: 1,
};

// useProducts
{ status: 'loading' | 'success' | 'error', products: Product[], error: string | null }

// useCart
{ p1: 2, p14: 1 }
```

Senior note: derive, don't store.

- `filtered` , `categoryCounts` , `pageItems` , `totalPages` , current `page` (clamped), `cart lines` , `count` , `subtotal` : all derived with `useMemo` or plain calls. Storing any of them means keeping two copies in sync.
- The cart stores **only ids and quantities**, never price or title: prices change; the catalog is the truth.

- Ownership: URL owns filters, the API owns the catalog, localStorage owns the cart, components own only drafts (search text, price inputs) and UI flags (`cartOpen`).

5. Design the data structures and contracts

Product (stable string ids, so cart keys stay valid across reloads and re-sorts)

```
{
  id: 'p1',
  title: 'Wireless Earbuds',
  category: 'electronics',
  icon: '🎧',
  price: 1139,      // integer rupees: no floating point money bugs
  rating: 3.7,
  stock: 5,         // 0 = out of stock
  createdAt: 1,     // used by "newest"
}
```

API contract (mocked)

```
fetchProducts({ delay = 600, failRate = 0.1 }) -> Promise<Product[]>
  resolve: [ { id, title, category, icon, price, rating, stock, createdAt }, ... ]
  reject:  Error('Could not load products. Please try again.')
```

For a server-side version the contract becomes `GET /api/products?q=shoe&cat=books&sort=price-asc&page=2` returning `{ items, total, facets: { books: 8, ... } }`, and the same `serializeFilters` builds the query string.

URL contract

Param	Example	Default (left out)	Invalid value
`q`	`q=clean+code`	empty	n/a, capped at 100 chars
`cat`	`cat=electronics%2Chome`	none	unknown names dropped, fixed order
`min` , `max`	`min=500&max=2000`	none	non-numbers / negatives dropped, swapped if min > max
`rating`	`rating=4`	0 (any)	anything not 1-4 becomes 0
`stock`	`stock=1`	off	anything but `1` is off
`sort`	`sort=price-asc`	relevance	unknown sort becomes relevance
`page`	`page=3`	1	non-positive / non-integer becomes 1

Cart actions

```
{ type: 'add', id, stock }
{ type: 'setQty', id, qty, stock }   // clamps 0..stock, 0 removes
{ type: 'remove', id }
{ type: 'clear' }
{ type: 'sync', items }              // from another tab
```

6. Trace the data flow

User ticks "Books" while on page 3, sorted by price

```
Click checkbox "books"
        |
        v
FilterPanel -> onChange({ categories: [...current, 'books'] })
        |
        v
setFilters(patch)
  current = parseFilters(location.search)      // live URL, not a stale closure
  next    = { ...current, page: 1, ...patch }  // filter change resets page
  qs      = serializeFilters(next)             // "?cat=books&sort=price-asc"
  history.pushState(null, '', qs)
  dispatchEvent(new Event('urlchange'))
        |
        v
useSyncExternalStore sees a new location.search -> App re-renders
        |
        v
filters  = parseFilters(search)                          (useMemo)
filtered = applyFilters(products, filters)               (useMemo: filter + sort)
counts   = getCategoryCounts(products, filters)          (useMemo: ignores category)
{ pageItems, page, totalPages } = paginate(filtered, 1)
        |
        v
Grid shows 8 books sorted by price, "8 products" announced (aria-live),
checkbox ticked, Pagination hidden (1 page)

Browser Back -> popstate -> same path in reverse -> page 3 of the old view returns.
```

7. Important technical challenges

Challenge 1: The URL is the state (not a copy of it)

Problem. The common approach is `useState` for filters plus an effect that writes them to the URL, plus another effect that reads the URL on load. Now there are two sources of truth. Back button changes the URL but not the state; a refresh with a typo in the URL crashes a `.map`; two quick updates overwrite each other because the effect reads a stale state.

Solution. Never store filters in React state. Read `location.search` with `useSyncExternalStore` (subscribe to `popstate` plus our own event, because `pushState` fires none), parse it into a filters object, and write back through one `setFilters(patch)` that reads the **live** URL. Parsing validates everything, so any URL is safe.

```
export function useUrlFilters() {
  const search = useSyncExternalStore(subscribe, () => window.location.search);
  const filters = useMemo(() => parseFilters(search), [search]);

  const setFilters = useCallback((patch, { replace = false } = {}) => {
    const current = parseFilters(window.location.search);
    const next = { ...current, page: 1, ...patch };
    const qs = serializeFilters(next);
    if (qs === window.location.search) return;
    window.history[replace ? 'replaceState' : 'pushState'](null, '', qs || window.location.pathname);
    window.dispatchEvent(new Event('urlchange'));
  }, []);

  return [filters, setFilters];
}
```

`serializeFilters` leaves out defaults and writes categories in a fixed order, so the same filters always give the same URL (good for sharing, caching and analytics).

Challenge 2: Debounced search box that still follows the URL

Problem. Writing every keystroke to the URL re-filters on every key and fills history. Debouncing needs a local draft. But the draft must also update when the URL changes from outside (Back, "Clear search"). Two classic bugs:

- A `useEffect(() => setText(value), [value])` re-syncs one render late and flickers.
- Trimming the query when parsing: the user types "running ", the debounce commits, the URL comes back as "running", the box overwrites the user's space, and they end up typing "runnings".

Solution. Keep a draft in local state and resync it **during render** when the prop changes (React's "adjust state when a prop changes" pattern). Never trim the stored query; trim only when matching.

```
const [text, setText] = useState(value);
const [synced, setSynced] = useState(value);
if (value !== synced) { setSynced(value); setText(value); } // Back / Clear

useEffect(() => {
  if (text === value) return;
  const t = setTimeout(() => onCommit(text), 300);  // replaceState, no history spam
  return () => clearTimeout(t);
}, [text, value, onCommit]);
```

The price inputs have the same problem in another shape: committing each keystroke would filter on "1" while the user types "1500", and auto-swap min / max mid-typing. So price uses a small form with drafts and

a **Go** button, remounted with `key={min-max}` when the URL range changes.

Challenge 3: Derived pipeline: filter, facet counts, sort, paginate

Problem. Four things depend on the same filters. Category counts that use the category filter itself are wrong: tick "Books" and every other category shows 0, so the user can't add a second category. Page 4 of 4, then a filter leaves 1 page: the grid is empty. Sorting with `.sort()` on the products array mutates the API data.

Solution. One `matches(product, filters, skipCategory)` predicate shared by both the list and the counts; counts skip the category check (standard faceted search). `filter()` makes a copy, so sorting that copy is safe. `paginate` clamps the page, and every filter change resets `page` to 1.

```
export function getCategoryCounts(products, f) {
  const counts = Object.fromEntries(CATEGORIES.map((c) => [c, 0]));
  for (const p of products) if (matches(p, f, true)) counts[p.category] += 1;
  return counts;
}

export function paginate(items, page) {
  const totalPages = Math.max(1, Math.ceil(items.length / PAGE_SIZE));
  const current = Math.min(page, totalPages);
  return { pageItems: items.slice((current - 1) * PAGE_SIZE, current * PAGE_SIZE), page: current,
totalPages };
}
```

Challenge 4: Cart that is persistent, stock-capped and never stale

Problem. The cart is saved on Monday with 9 earbuds at ₹999. On Tuesday the price is ₹1,139 and only 5 are left. If the cart stored prices, the subtotal is wrong; if it trusts the saved qty, the user orders stock that doesn't exist. localStorage can also hold junk (old format, hand edits), and a second tab can change the cart.

Solution. Store only `{ id: qty }`. A pure reducer clamps every change to `0..stock` and returns the **same object** when nothing changes (no useless re-render or write). `loadCart` keeps only positive integers. `getCartSummary` joins the saved cart with the fresh catalog: drops deleted products, clamps to current stock and flags the line. A `storage` listener adopts changes from other tabs.

```
function setQty(items, id, qty, stock) {
  const next = Math.min(Math.max(0, Math.floor(qty)), stock);
  if (next === (items[id] ?? 0)) return items;
  if (next === 0) { const rest = { ...items }; delete rest[id]; return rest; }
  return { ...items, [id]: next };
}

export function getCartSummary(items, productsById) {
  const lines = [];
  for (const [id, saved] of Object.entries(items)) {
    const product = productsById[id];
```

```
    if (!product) continue;
    const qty = Math.min(saved, product.stock);
    lines.push({ product, qty, adjusted: qty < saved, lineTotal: qty * product.price });
  }
  return { lines, count: lines.reduce((n, l) => n + l.qty, 0), subtotal: lines.reduce((s, l) => s +
l.lineTotal, 0) };
}
```

8. Implement in the correct order

#	Step	Working result
1	Mock API with deterministic catalog	`fetchProducts()` resolves 48 products
2	`useProducts` + grid of cards	Products on screen with skeleton and retry
3	`applyFilters` with search + category, hard-coded filters	Grid filters correctly
4	`parseFilters` / `serializeFilters` + `useUrlFilters`	Changing the URL by hand changes the grid
5	FilterPanel wired to `setFilters`	Checkboxes update URL; Back / Forward works
6	Price, rating, in stock, sort	All filters in the URL
7	`SearchBox` with draft + debounce	Typing is smooth, URL updated after 300 ms
8	Facet counts + pagination + page reset / clamp	Counts correct, no empty pages
9	`cartReducer` + Add / stepper on cards	Cart works in memory, stock-capped
10	Persistence, `loadCart`, cross-tab sync	Cart survives refresh
11	CartDrawer with summary, focus, Escape	Full cart UI
12	Empty / error / invalid URL / clamped cart polish	All states in section 9 covered

9. Cover all states and edge cases

State / edge case	Handling
Initial load	Skeleton cards, `aria-busy`
API error	Message + Retry (resets to loading, new attempt)
Late response from an old attempt	`ignore` flag in the effect cleanup
Success, nothing matches	"No products match" + "Clear search and filters"
Search is only spaces	Not written to the URL; treated as no search
Search with trailing space while typing	Kept as typed; trimmed only when matching
Back / Forward	`popstate` re-renders from URL; search box and price inputs resync

State / edge case	Handling
Invalid params (`sort=toString` , `rating=9` , `page=-2` , `cat=junk`)	Fall back to defaults, never crash
`min` > `max`	Swapped (on submit and when parsing a hand-edited URL)
Negative price	Ignored
`page` beyond last page	Clamped to the last page
Any filter change while on page 3	Page resets to 1
Same URL written again	No history entry (string compare)
Out of stock product	Disabled "Add to cart", red "Out of stock"
Click + at stock limit	Button disabled; reducer clamps anyway
Qty goes to 0	Line removed
Saved qty > current stock	Clamped, "Only N left" note
Saved product no longer exists	Line dropped
Corrupt localStorage / storage blocked	Empty cart; write errors swallowed, cart works in memory
Cart changed in another tab	`storage` event, `sync` action
Drawer open	Focus on Close; Escape / backdrop close; focus returns to cart button
1 product / 1 item	Singular text ("1 product", "1 item")

10. Final structure

```
_73_ecommerce_listing_cart_filters/
├── index.html
├── package.json
├── vite.config.js
└── src/
    ├── main.jsx
    ├── index.css
    ├── App.jsx                    # wires hooks, derives list, renders states
    ├── api/
    │   └── productsApi.js         # catalog + fetchProducts mock
    ├── utils/
    │   ├── filters.js             # parse, serialize, filter, counts, paginate
    │   ├── cart.js                # reducer, loadCart, getCartSummary
    │   └── format.js              # INR formatter
    ├── hooks/
    │   ├── useUrlFilters.js       # URL as state
    │   ├── useProducts.js         # fetch status machine
    │   └── useCart.js             # reducer + localStorage + cross-tab
```

```
    └── components/
        ├── SearchBox.jsx
        ├── FilterPanel.jsx
        ├── PriceFilter.jsx
        ├── ProductCard.jsx
        ├── QtyStepper.jsx
        ├── Pagination.jsx
        └── CartDrawer.jsx
```

11. Full working code

Setup: `npm create vite@latest` (React, JavaScript), then `npm i tailwindcss @tailwindcss/vite`.

vite.config.js

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react-swc';
import tailwindcss from '@tailwindcss/vite';

export default defineConfig({
  plugins: [react(), tailwindcss()],
});
```

src/index.css

```
@import "tailwindcss";
```

src/main.jsx

```
import { StrictMode } from 'react';
import { createRoot } from 'react-dom/client';
import './index.css';
import App from './App';

createRoot(document.getElementById('root')).render(
  <StrictMode>
    <App />
  </StrictMode>
);
```

src/api/productsApi.js

```
export const CATEGORIES = ['electronics', 'fashion', 'home', 'books', 'sports', 'beauty'];

const NAMES = {
  electronics: ['Wireless Earbuds', 'Bluetooth Speaker', 'Smart Watch', 'Power Bank', 'USB-C Hub',
```

```
'Mechanical Keyboard', 'Gaming Mouse', 'HD Webcam'],
  fashion: ['Running Shoes', 'Denim Jacket', 'Cotton T-Shirt', 'Leather Belt', 'Canvas Sneakers', 'Zip Hoodie', 'Sunglasses', 'Laptop Backpack'],
  home: ['Ceramic Mug Set', 'Table Lamp', 'Throw Pillow', 'Wall Clock', 'Bedsheet Set', 'Storage Box', 'Plant Pot', 'Door Mat'],
  books: ['Clean Code', 'Atomic Habits', 'The Pragmatic Programmer', 'Deep Work', 'Refactoring', 'Sapiens', 'Ikigai', 'Design Patterns'],
  sports: ['Yoga Mat', 'Dumbbell Pair', 'Skipping Rope', 'Football', 'Badminton Racket', 'Cricket Bat', 'Water Bottle', 'Resistance Bands'],
  beauty: ['Face Wash', 'Sunscreen SPF 50', 'Lip Balm', 'Hair Serum', 'Moisturiser', 'Perfume', 'Beard Oil', 'Face Mask'],
};

const ICONS = { electronics: '🎧', fashion: '👟', home: '🛋️', books: '📚', sports: '⚽', beauty: '💄' };

// Deterministic catalog, so the same URL always shows the same results.
export const PRODUCTS = CATEGORIES.flatMap((category, c) =>
  NAMES[category].map((title, i) => {
    const n = c * 8 + i + 1;
    return {
      id: `p${n}`,
      title,
      category,
      icon: ICONS[category],
      price: Math.round((199 + ((n * 937) % 4800)) / 10) * 10 - 1, // ₹209 .. ₹4999
      rating: (30 + ((n * 7) % 21)) / 10, // 3.0 .. 5.0
      stock: (n * 5) % 7, // 0 = out of stock
      createdAt: n,
    };
  })
);

// Mock API. shortcut: returns the whole catalog and filters on the client;
// with thousands of products send serializeFilters(filters) as the query string instead.
export function fetchProducts({ delay = 600, failRate = 0.1 } = {}) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (Math.random() < failRate) reject(new Error('Could not load products. Please try again.'));
      else resolve(structuredClone(PRODUCTS));
    }, delay);
  });
}
```

src/utils/filters.js

```
import { CATEGORIES } from '../api/productsApi.js';

export const PAGE_SIZE = 12;

export const SORTS = {
  relevance: { label: 'Relevance', cmp: null },
  'price-asc': { label: 'Price: low to high', cmp: (a, b) => a.price - b.price },
  'price-desc': { label: 'Price: high to low', cmp: (a, b) => b.price - a.price },
  rating: { label: 'Top rated', cmp: (a, b) => b.rating - a.rating },
  newest: { label: 'Newest first', cmp: (a, b) => b.createdAt - a.createdAt },
};

export const DEFAULT_FILTERS = {
  q: '',
  categories: [],
  min: null,
  max: null,
  rating: 0,
  inStock: false,
  sort: 'relevance',
  page: 1,
};

const toPrice = (v) => {
  if (v === null || v.trim() === '') return null;
  const n = Number(v);
  return Number.isFinite(n) && n >= 0 ? n : null;
};

// The URL is user input: anything invalid falls back to its default.
export function parseFilters(search) {
  const p = new URLSearchParams(search);
  let min = toPrice(p.get('min'));
  let max = toPrice(p.get('max'));
  if (min !== null && max !== null && min > max) [min, max] = [max, min];
  const picked = new Set((p.get('cat') ?? '').split(','));
  const rating = Number(p.get('rating'));
  const page = Number(p.get('page'));
  const sort = p.get('sort');
  return {
    q: (p.get('q') ?? '').slice(0, 100), // not trimmed: the user may still be typing "running "
    categories: CATEGORIES.filter((c) => picked.has(c)),
    min,
    max,
    rating: [1, 2, 3, 4].includes(rating) ? rating : 0,
    inStock: p.get('stock') === '1',
    sort: Object.hasOwn(SORTS, sort ?? '') ? sort : 'relevance',
```

```
    page: Number.isInteger(page) && page > 0 ? page : 1,
  };
}

// Defaults are left out, so a clean listing has a clean URL.
export function serializeFilters(f) {
  const p = new URLSearchParams();
  if (f.q.trim()) p.set('q', f.q);
  const cats = CATEGORIES.filter((c) => f.categories.includes(c)); // fixed order: same filters, same
URL
  if (cats.length) p.set('cat', cats.join(','));
  if (f.min !== null) p.set('min', f.min);
  if (f.max !== null) p.set('max', f.max);
  if (f.rating) p.set('rating', f.rating);
  if (f.inStock) p.set('stock', '1');
  if (f.sort !== 'relevance') p.set('sort', f.sort);
  if (f.page > 1) p.set('page', f.page);
  const qs = p.toString();
  return qs ? `?${qs}` : '';
}

function matches(product, f, skipCategory = false) {
  const q = f.q.trim().toLowerCase();
  return (
    (!q || product.title.toLowerCase().includes(q) || product.category.includes(q)) &&
    (skipCategory || !f.categories.length || f.categories.includes(product.category)) &&
    (f.min === null || product.price >= f.min) &&
    (f.max === null || product.price <= f.max) &&
    product.rating >= f.rating &&
    (!f.inStock || product.stock > 0)
  );
}

export function applyFilters(products, f) {
  const out = products.filter((p) => matches(p, f));
  const { cmp } = SORTS[f.sort];
  return cmp ? out.sort(cmp) : out; // filter() made a copy, so sorting in place is safe
}

// Counts ignore the category filter itself, so ticking "books" doesn't zero out "home".
export function getCategoryCounts(products, f) {
  const counts = Object.fromEntries(CATEGORIES.map((c) => [c, 0]));
  for (const p of products) if (matches(p, f, true)) counts[p.category] += 1;
  return counts;
}

export function paginate(items, page) {
```

```
  const totalPages = Math.max(1, Math.ceil(items.length / PAGE_SIZE));
  const current = Math.min(page, totalPages); // page=9 in an old link shows the last page
  return {
    pageItems: items.slice((current - 1) * PAGE_SIZE, current * PAGE_SIZE),
    page: current,
    totalPages,
  };
}

export const countActiveFilters = (f) =>
  f.categories.length + (f.min !== null) + (f.max !== null) + (f.rating > 0) + f.inStock;
```

src/utils/cart.js

```
export const CART_KEY = 'cart:v1';

// Cart state is { [productId]: qty }. Prices are looked up from the catalog, never stored.
export function cartReducer(items, action) {
  switch (action.type) {
    case 'add':
      return setQty(items, action.id, (items[action.id] ?? 0) + 1, action.stock);
    case 'setQty':
      return setQty(items, action.id, action.qty, action.stock);
    case 'remove':
      return setQty(items, action.id, 0, 0);
    case 'clear':
      return {};
    case 'sync':
      return action.items;
    default:
      return items;
  }
}

function setQty(items, id, qty, stock) {
  const next = Math.min(Math.max(0, Math.floor(qty)), stock);
  if (next === (items[id] ?? 0)) return items; // no change, same reference, no re-render
  if (next === 0) {
    const rest = { ...items };
    delete rest[id];
    return rest;
  }
  return { ...items, [id]: next };
}

// localStorage can hold anything (old versions, hand edits): keep only positive integer quantities.
export function loadCart(storage = globalThis.localStorage) {
```

```
  try {
    const raw = JSON.parse(storage.getItem(CART_KEY));
    if (!raw || typeof raw !== 'object' || Array.isArray(raw)) return {};
    return Object.fromEntries(Object.entries(raw).filter(([, q]) => Number.isInteger(q) && q > 0));
  } catch {
    return {};
  }
}

// Join the saved cart with the fresh catalog: drop deleted products, clamp to current stock.
export function getCartSummary(items, productsById) {
  const lines = [];
  for (const [id, saved] of Object.entries(items)) {
    const product = productsById[id];
    if (!product) continue;
    const qty = Math.min(saved, product.stock);
    lines.push({ product, qty, adjusted: qty < saved, lineTotal: qty * product.price });
  }
  return {
    lines,
    count: lines.reduce((n, l) => n + l.qty, 0),
    subtotal: lines.reduce((sum, l) => sum + l.lineTotal, 0),
  };
}
```

src/utils/format.js

```
const inr = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR', maximumFractionDigits:
0 });

export const formatPrice = (n) => inr.format(n);
```

src/hooks/useUrlFilters.js

```
import { useCallback, useMemo, useSyncExternalStore } from 'react';
import { parseFilters, serializeFilters } from '../utils/filters';

// pushState/replaceState fire no event, so we announce our own changes.
const URL_CHANGE = 'urlchange';

function subscribe(callback) {
  window.addEventListener('popstate', callback); // back / forward
  window.addEventListener(URL_CHANGE, callback);
  return () => {
    window.removeEventListener('popstate', callback);
    window.removeEventListener(URL_CHANGE, callback);
```

```
  };
}

const getSearch = () => window.location.search;

// The URL is the single source of truth for filters: no copy in React state.
export function useUrlFilters() {
  const search = useSyncExternalStore(subscribe, getSearch);
  const filters = useMemo(() => parseFilters(search), [search]);

  const setFilters = useCallback((patch, { replace = false } = {}) => {
    // Read the live URL, not the render's `filters`, so two quick updates never overwrite each other.
    const current = parseFilters(window.location.search);
    const next = { ...current, page: 1, ...patch }; // any filter change goes back to page 1
    const qs = serializeFilters(next);
    if (qs === window.location.search) return;
    window.history[replace ? 'replaceState' : 'pushState'](null, '', qs || window.location.pathname);
    window.dispatchEvent(new Event(URL_CHANGE));
  }, []);

  return [filters, setFilters];
}
```

src/hooks/useProducts.js

```
import { useEffect, useState } from 'react';
import { fetchProducts } from '../api/productsApi';

export function useProducts() {
  const [state, setState] = useState({ status: 'loading', products: [], error: null });
  const [attempt, setAttempt] = useState(0);

  useEffect(() => {
    let ignore = false; // a late response from an old attempt must not win
    fetchProducts().then(
      (products) => !ignore && setState({ status: 'success', products, error: null }),
      (err) => !ignore && setState({ status: 'error', products: [], error: err.message })
    );
    return () => {
      ignore = true;
    };
  }, [attempt]);

  const retry = () => {
    setState({ status: 'loading', products: [], error: null });
    setAttempt((a) => a + 1);
  };
```

```
  return { ...state, retry };
}
```

src/hooks/useCart.js

```
import { useEffect, useReducer } from 'react';
import { CART_KEY, cartReducer, loadCart } from '../utils/cart';

export function useCart() {
  const [items, dispatch] = useReducer(cartReducer, undefined, () => loadCart());

  useEffect(() => {
    try {
      localStorage.setItem(CART_KEY, JSON.stringify(items));
    } catch {
      // Quota full or storage blocked: the cart still works for this visit.
    }
  }, [items]);

  // Another tab changed the cart: take its copy. (Writing the same value back fires no event, so no
loop.)
  useEffect(() => {
    const onStorage = (e) => {
      if (e.key === CART_KEY) dispatch({ type: 'sync', items: loadCart() });
    };
    window.addEventListener('storage', onStorage);
    return () => window.removeEventListener('storage', onStorage);
  }, []);

  return [items, dispatch];
}
```

src/components/SearchBox.jsx

```
import { useEffect, useState } from 'react';

// Keeps a fast local draft while typing and writes it to the URL 300 ms after the user stops.
export default function SearchBox({ value, onCommit }) {
  const [text, setText] = useState(value);
  const [synced, setSynced] = useState(value);

  // The URL changed from outside (back button, clear): show the new value.
  if (value !== synced) {
    setSynced(value);
    setText(value);
```

```
  }

  useEffect(() => {
    if (text === value) return;
    const t = setTimeout(() => onCommit(text), 300);
    return () => clearTimeout(t);
  }, [text, value, onCommit]);

  return (
    <form
      role="search"
      className="flex-1"
      onSubmit={(e) => {
        e.preventDefault();
        onCommit(text);
      }}
    >
      <label htmlFor="search" className="sr-only">
        Search products
      </label>
      <input
        id="search"
        type="search"
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Search products"
        className="w-full rounded-md border border-slate-300 px-3 py-2 text-sm outline-none
focus:border-indigo-500 focus:ring-2 focus:ring-indigo-200"
      />
    </form>
  );
}
```

src/components/PriceFilter.jsx

```
import { useState } from 'react';

const toValue = (s) => (s === '' ? null : Math.max(0, Number(s)));

// Draft inputs applied on submit: committing every keystroke would filter on "1" while the user types
"1500".
// The parent remounts this with a new `key` when the URL range changes, which resets the drafts.
export default function PriceFilter({ min, max, onApply }) {
  const [draftMin, setDraftMin] = useState(min ?? '');
  const [draftMax, setDraftMax] = useState(max ?? '');

  return (
```

```
    <form
      className="flex items-end gap-2"
      onSubmit={(e) => {
        e.preventDefault();
        let min = toValue(draftMin);
        let max = toValue(draftMax);
        if (min !== null && max !== null && min > max) [min, max] = [max, min]; // "3000 to 1000" means
1000 to 3000
        onApply({ min, max });
      }}
    >
      <label className="text-xs text-slate-500">
        Min ₹
        <input
          type="number"
          min="0"
          inputMode="numeric"
          value={draftMin}
          onChange={(e) => setDraftMin(e.target.value)}
          className="mt-1 w-full rounded border border-slate-300 px-2 py-1 text-sm text-slate-900"
        />
      </label>
      <label className="text-xs text-slate-500">
        Max ₹
        <input
          type="number"
          min="0"
          inputMode="numeric"
          value={draftMax}
          onChange={(e) => setDraftMax(e.target.value)}
          className="mt-1 w-full rounded border border-slate-300 px-2 py-1 text-sm text-slate-900"
        />
      </label>
      <button type="submit" className="rounded bg-slate-800 px-2 py-1 text-sm text-white hover:bg-
slate-700">
        Go
      </button>
    </form>
  );
}
```

src/components/FilterPanel.jsx

```
import { CATEGORIES } from '../api/productsApi';
import { countActiveFilters } from '../utils/filters';
import PriceFilter from './PriceFilter';
```

```
export default function FilterPanel({ filters, counts, onChange }) {
  const active = countActiveFilters(filters);

  const toggleCategory = (c) =>
    onChange({
      categories: filters.categories.includes(c)
        ? filters.categories.filter((x) => x !== c)
        : [...filters.categories, c],
    });

  return (
    <aside className="space-y-5 rounded-lg border border-slate-200 bg-white p-4 md:self-start" aria-
label="Filters">
      <div className="flex items-center justify-between">
        <h2 className="font-semibold">Filters {active > 0 && <span className="text-slate-500">
({active})</span>}</h2>
        {active > 0 && (
          <button
            type="button"
            onClick={() => onChange({ categories: [], min: null, max: null, rating: 0, inStock: false
})}
            className="text-sm text-indigo-600 hover:underline"
          >
            Clear all
          </button>
        )}
      </div>

      <fieldset>
        <legend className="mb-2 text-sm font-medium">Category</legend>
        {CATEGORIES.map((c) => (
          <label key={c} className="flex items-center gap-2 py-0.5 text-sm capitalize">
            <input type="checkbox" checked={filters.categories.includes(c)} onChange={() =>
toggleCategory(c)} />
            {c}
            <span className="ml-auto text-xs text-slate-400">{counts[c]}</span>
          </label>
        ))}
      </fieldset>

      <fieldset>
        <legend className="mb-2 text-sm font-medium">Price</legend>
        <PriceFilter
          key={`${filters.min}-${filters.max}`}
          min={filters.min}
          max={filters.max}
          onApply={onChange}
```

```
        />
      </fieldset>

      <label className="block text-sm font-medium">
        Customer rating
        <select
          value={filters.rating}
          onChange={(e) => onChange({ rating: Number(e.target.value) })}
          className="mt-1 w-full rounded border border-slate-300 px-2 py-1 font-normal"
        >
          <option value={0}>Any</option>
          {[4, 3, 2, 1].map((r) => (
            <option key={r} value={r}>
              {r}★ & up
            </option>
          ))}
        </select>
      </label>

      <label className="flex items-center gap-2 text-sm">
        <input type="checkbox" checked={filters.inStock} onChange={(e) => onChange({ inStock:
e.target.checked })} />
        In stock only
      </label>
    </aside>
  );
}
```

src/components/QtyStepper.jsx

```
export default function QtyStepper({ qty, max, label, onChange }) {
  const btn = 'h-8 w-8 rounded border border-slate-300 text-lg leading-none hover:bg-slate-100
disabled:opacity-40';
  return (
    <div className="flex items-center gap-2">
      <button type="button" className={btn} onClick={() => onChange(qty - 1)} aria-label={`Decrease
quantity of ${label}`}>
        –
      </button>
      <span className="w-6 text-center text-sm font-medium" aria-live="polite">
        {qty}
      </span>
      <button
        type="button"
        className={btn}
        onClick={() => onChange(qty + 1)}
        disabled={qty >= max}
```

```
        aria-label={`Increase quantity of ${label}`}
      >
        +
      </button>
    </div>
  );
}
```

src/components/ProductCard.jsx

```
import { memo } from 'react';
import { formatPrice } from '../utils/format';
import QtyStepper from './QtyStepper';

// memo: changing one card's quantity re-renders only that card.
function ProductCard({ product, qty, dispatch }) {
  const { id, title, category, icon, price, rating, stock } = product;
  const setQty = (q) => dispatch({ type: 'setQty', id, qty: q, stock });

  return (
    <li className="flex flex-col rounded-lg border border-slate-200 bg-white p-3">
      <div className="grid h-28 place-items-center rounded-md bg-slate-100 text-5xl" aria-
hidden="true">
        {icon}
      </div>
      <h3 className="mt-2 text-sm font-semibold">{title}</h3>
      <p className="text-xs capitalize text-slate-500">{category}</p>
      <p className="mt-1 text-xs">
        <span className="rounded bg-emerald-600 px-1.5 py-0.5 font-medium text-white">
{rating.toFixed(1)} ★</span>
      </p>
      <p className="mt-2 font-bold">{formatPrice(price)}</p>
      <p className={`text-xs ${stock === 0 ? 'text-red-600' : stock <= 2 ? 'text-amber-600' : 'text-
slate-500'}`}>
        {stock === 0 ? 'Out of stock' : stock <= 2 ? `Only ${stock} left` : 'In stock'}
      </p>
      <div className="mt-auto pt-3">
        {qty > 0 ? (
          <QtyStepper qty={qty} max={stock} label={title} onChange={setQty} />
        ) : (
          <button
            type="button"
            disabled={stock === 0}
            onClick={() => dispatch({ type: 'add', id, stock })}
            className="w-full rounded-md bg-indigo-600 py-1.5 text-sm font-medium text-white hover:bg-
indigo-500 disabled:bg-slate-300"
          >
```

```
            Add to cart
          </button>
        )}
      </div>
    </li>
  );
}

export default memo(ProductCard);
```

src/components/Pagination.jsx

```
export default function Pagination({ page, totalPages, onPage }) {
  if (totalPages <= 1) return null;
  const btn = 'min-w-9 rounded border border-slate-300 px-3 py-1 text-sm hover:bg-slate-100
disabled:opacity-40';

  // shortcut: renders every page number; add a window with "…" when there are more than ~7 pages.
  return (
    <nav aria-label="Pagination" className="mt-6 flex justify-center gap-1">
      <button type="button" className={btn} disabled={page === 1} onClick={() => onPage(page - 1)}>
        Prev
      </button>
      {Array.from({ length: totalPages }, (_, i) => i + 1).map((n) => (
        <button
          key={n}
          type="button"
          onClick={() => onPage(n)}
          aria-current={n === page ? 'page' : undefined}
          className={`${btn} ${n === page ? 'border-indigo-600 bg-indigo-600 text-white hover:bg-
indigo-600' : ''}`}
        >
          {n}
        </button>
      ))}
      <button type="button" className={btn} disabled={page === totalPages} onClick={() => onPage(page +
1)}>
        Next
      </button>
    </nav>
  );
}
```

src/components/CartDrawer.jsx

```
import { useEffect, useRef } from 'react';
import { formatPrice } from '../utils/format';
import QtyStepper from './QtyStepper';

export default function CartDrawer({ summary, dispatch, onClose }) {
  const closeRef = useRef(null);

  // Focus the drawer on open, close on Escape, give focus back to the cart button on close.
  useEffect(() => {
    const opener = document.activeElement;
    closeRef.current.focus();
    const onKey = (e) => e.key === 'Escape' && onClose();
    window.addEventListener('keydown', onKey);
    return () => {
      window.removeEventListener('keydown', onKey);
      opener?.focus();
    };
  }, [onClose]);

  const { lines, count, subtotal } = summary;

  return (
    <div className="fixed inset-0 z-20">
      <div className="absolute inset-0 bg-black/40" onClick={onClose} aria-hidden="true" />
      <aside
        role="dialog"
        aria-modal="true"
        aria-labelledby="cart-title"
        className="absolute right-0 top-0 flex h-full w-full max-w-sm flex-col bg-white shadow-xl"
      >
        <div className="flex items-center justify-between border-b border-slate-200 p-4">
          <h2 id="cart-title" className="text-lg font-semibold">
            Your cart ({count})
          </h2>
          <button ref={closeRef} type="button" onClick={onClose} aria-label="Close cart"
className="rounded px-2 text-2xl hover:bg-slate-100">
            ×
          </button>
        </div>

        {lines.length === 0 ? (
          <p className="p-6 text-center text-slate-500">Your cart is empty.</p>
        ) : (
          <ul className="flex-1 divide-y divide-slate-100 overflow-y-auto px-4">
            {lines.map(({ product, qty, adjusted, lineTotal }) => (
              <li key={product.id} className="flex gap-3 py-3">
                <span className="grid h-12 w-12 shrink-0 place-items-center rounded bg-slate-100 text-
```

```
2xl" aria-hidden="true">
                  {product.icon}
                </span>
                <div className="flex-1">
                  <p className="text-sm font-medium">{product.title}</p>
                  <p className="text-xs text-slate-500">{formatPrice(product.price)} each</p>
                  {adjusted && (
                    <p className="text-xs text-amber-600">
                      {qty === 0 ? 'Now out of stock' : `Only ${product.stock} left, quantity updated`}
                    </p>
                  )}
                  <div className="mt-2 flex items-center justify-between">
                    <QtyStepper
                      qty={qty}
                      max={product.stock}
                      label={product.title}
                      onChange={(q) => dispatch({ type: 'setQty', id: product.id, qty: q, stock:
product.stock })}
                    />
                    <button
                      type="button"
                      onClick={() => dispatch({ type: 'remove', id: product.id })}
                      className="text-xs text-red-600 hover:underline"
                    >
                      Remove
                    </button>
                  </div>
                </div>
                <p className="text-sm font-semibold">{formatPrice(lineTotal)}</p>
              </li>
            ))}
          </ul>
        )}

        <div className="mt-auto border-t border-slate-200 p-4">
          <p className="flex justify-between font-semibold">
            <span>Subtotal</span>
            <span>{formatPrice(subtotal)}</span>
          </p>
          {lines.length > 0 && (
            <button type="button" onClick={() => dispatch({ type: 'clear' })} className="mt-3 text-sm
text-slate-500 hover:underline">
              Clear cart
            </button>
          )}
        </div>
      </aside>
```

```
    </div>
  );
}
```

src/App.jsx

```
import { useCallback, useMemo, useState } from 'react';
import { useUrlFilters } from './hooks/useUrlFilters';
import { useProducts } from './hooks/useProducts';
import { useCart } from './hooks/useCart';
import { applyFilters, getCategoryCounts, paginate, SORTS } from './utils/filters';
import { getCartSummary } from './utils/cart';
import SearchBox from './components/SearchBox';
import FilterPanel from './components/FilterPanel';
import ProductCard from './components/ProductCard';
import Pagination from './components/Pagination';
import CartDrawer from './components/CartDrawer';

export default function App() {
  const [filters, setFilters] = useUrlFilters();
  const { status, products, error, retry } = useProducts();
  const [cart, dispatch] = useCart();
  const [cartOpen, setCartOpen] = useState(false);

  // Everything below is derived: nothing about the listing is stored twice.
  const filtered = useMemo(() => applyFilters(products, filters), [products, filters]);
  const counts = useMemo(() => getCategoryCounts(products, filters), [products, filters]);
  const { pageItems, page, totalPages } = paginate(filtered, filters.page);
  const byId = useMemo(() => Object.fromEntries(products.map((p) => [p.id, p])), [products]);
  const summary = useMemo(() => getCartSummary(cart, byId), [cart, byId]);

  const commitSearch = useCallback((q) => setFilters({ q }, { replace: true }), [setFilters]);
  const closeCart = useCallback(() => setCartOpen(false), []);
  const clearFilters = () => setFilters({ q: '', categories: [], min: null, max: null, rating: 0,
inStock: false });

  return (
    <div className="min-h-screen bg-slate-50 text-slate-900">
      <header className="sticky top-0 z-10 border-b border-slate-200 bg-white">
        <div className="mx-auto flex max-w-6xl items-center gap-3 px-4 py-3">
          <h1 className="text-lg font-bold text-indigo-600">ShopLite</h1>
          <SearchBox value={filters.q} onCommit={commitSearch} />
          <button
            type="button"
            onClick={() => setCartOpen(true)}
            aria-label={`Open cart, ${summary.count} ${summary.count === 1 ? 'item' : 'items'}`}
            className="relative rounded-md border border-slate-300 px-3 py-2 text-sm hover:bg-slate-
```

```
100"
          >
            🛒 Cart
            {summary.count > 0 && (
              <span className="absolute -right-2 -top-2 rounded-full bg-indigo-600 px-1.5 text-xs text-
white">{summary.count}</span>
            )}
          </button>
        </div>
      </header>

      <main className="mx-auto grid max-w-6xl gap-6 px-4 py-6 md:grid-cols-[230px_1fr]">
        <FilterPanel filters={filters} counts={counts} onChange={setFilters} />

        <section aria-labelledby="results-heading">
          <div className="mb-4 flex flex-wrap items-center justify-between gap-2">
            <h2 id="results-heading" className="text-sm text-slate-600" aria-live="polite">
              {status === 'success' ? `${filtered.length} ${filtered.length === 1 ? 'product' :
'products'}` : 'Products'}
            </h2>
            <label className="text-sm">
              Sort by{' '}
              <select
                value={filters.sort}
                onChange={(e) => setFilters({ sort: e.target.value })}
                className="rounded border border-slate-300 px-2 py-1"
              >
                {Object.entries(SORTS).map(([key, { label }]) => (
                  <option key={key} value={key}>
                    {label}
                  </option>
                ))}
              </select>
            </label>
          </div>

          {status === 'loading' && (
            <ul className="grid grid-cols-2 gap-4 lg:grid-cols-3" aria-busy="true" aria-label="Loading
products">
              {Array.from({ length: 6 }, (_, i) => (
                <li key={i} className="h-72 animate-pulse rounded-lg bg-slate-200" />
              ))}
            </ul>
          )}

          {status === 'error' && (
            <div role="alert" className="rounded-lg border border-red-200 bg-red-50 p-6 text-center">
```

```
              <p className="text-red-700">{error}</p>
              <button type="button" onClick={retry} className="mt-3 rounded-md bg-red-600 px-4 py-1.5
text-sm text-white">
                Retry
              </button>
            </div>
          )}

          {status === 'success' && filtered.length === 0 && (
            <div className="rounded-lg border border-dashed border-slate-300 p-10 text-center">
              <p className="text-slate-600">No products match these filters.</p>
              <button type="button" onClick={clearFilters} className="mt-3 text-sm text-indigo-600
hover:underline">
                Clear search and filters
              </button>
            </div>
          )}

          {status === 'success' && filtered.length > 0 && (
            <>
              <ul className="grid grid-cols-2 gap-4 lg:grid-cols-3">
                {pageItems.map((p) => (
                  <ProductCard key={p.id} product={p} qty={cart[p.id] ?? 0} dispatch={dispatch} />
                ))}
              </ul>
              <Pagination
                page={page}
                totalPages={totalPages}
                onPage={(n) => {
                  setFilters({ page: n });
                  window.scrollTo({ top: 0 });
                }}
              />
            </>
          )}
        </section>
      </main>

      {cartOpen && <CartDrawer summary={summary} dispatch={dispatch} onClose={closeCart} />}
    </div>
  );
}
```

12. Wrap-up

Edge cases handled

- Any URL is safe: invalid params fall back to defaults, unknown categories dropped, min / max swapped, page clamped.
- Back / Forward and refresh restore the exact view, including the search box text.
- Search is debounced and uses `replaceState`, so history isn't flooded; Enter commits immediately.
- Facet counts stay meaningful when a category is ticked.
- Cart: stock cap, 0 removes, junk storage ignored, prices from the fresh catalog, clamped saved quantities, cross-tab sync.
- Loading, error with retry, empty results, out of stock, accessible drawer.

Trade-offs to mention

Decision	Why	Cost
URL as the only filter state	Back / refresh / share work for free, no sync bugs	Every filter must be string-serialisable; parsing on each change (cheap)
No router library	Shows the platform mechanics (`pushState`, `popstate`)	In a real app use `useSearchParams` from React Router / Next
Client-side filtering	Simple, instant, fine for ~1000 items	Needs server filtering for a big catalog
Push for filters, replace for search	Back undoes a filter, not a keystroke	A finished search isn't its own history entry
Cart stores ids + qty only	Prices and stock always fresh	Needs the catalog loaded to show totals
Price applied with a Go button	No filtering on half-typed numbers	One extra click
Numbered pagination	Shareable `page=3`, predictable	Infinite scroll feels smoother on mobile

What to add with more time

- Server-side filtering with `AbortController` to cancel stale requests, and a cache keyed by the query string.
- Unit tests for `parseFilters` / `serializeFilters` round trip, `cartReducer`, `getCartSummary`; an RTL test for Back restoring filters.
- Filter chips above the grid ("Books ×"), mobile filter drawer, page number window with "…".
- Focus trap in the drawer (or native `<dialog>`), toast on "Added to cart".
- Server cart for logged-in users, merged with the guest cart on login; checkout re-validates price and stock on the server.
- Virtualised grid or infinite scroll for very large result sets.

13. Counter questions the interviewer may ask

REQUIREMENTS

Q1. Why put filters in the URL at all? Isn't useState enough?

useState is lost on refresh, can't be shared, and Back leaves the page instead of undoing the filter. In e-commerce the URL also matters for SEO and analytics (category landing pages). With the URL as the state, all

three work without extra code.

Q2. What should go in the URL and what shouldn't?

Anything that defines *what* the user is looking at: query, filters, sort, page. Not transient UI (is the cart drawer open, hover state, draft text) and never personal data. The cart is user data, so it goes to storage or the server, not the URL.

Q3. Should every filter change push a history entry?

Discrete choices (checkbox, sort, page) push, so Back undoes them one by one. Continuous input (typing) replaces, otherwise Back steps through "s", "sh", "sho". Some teams also replace for sort; it's a product decision worth asking about.

STATE DESIGN

Q4. Why not keep filters in state and sync them to the URL with an effect?

Two sources of truth. The effect runs after render, so there is a frame with mismatched state, and you need a second listener for Back. Deriving from the URL means there is nothing to sync: the URL changes, the component re-renders.

Q5. Why `useSyncExternalStore` instead of `useState` + a `popstate` listener?

The URL is an external mutable store; `useSyncExternalStore` is React's API for exactly that. It's tear-free in concurrent rendering and reads the current value during render, so the first render already has the right filters.

Q6. Why does `setFilters` read `window.location.search` instead of using `filters` from the render?

A closure over `filters` can be stale: if the debounced search fires right after a checkbox click in the same tick, it would write the old categories back. Reading the live URL always merges into the latest state. It's the URL version of a functional `setState(prev => ...)`.

Q7. Why does the cart store `{ id: qty }` and not the full product?

Price, title and stock change. Storing them means showing yesterday's price. Storing only ids keeps one source of truth (the catalog) and a tiny storage payload. The server must re-price at checkout anyway.

Q8. Why `useReducer` for the cart?

Several actions (add, setQty, remove, clear, sync) with rules (stock cap, 0 removes). A pure reducer keeps those rules in one testable function and lets cards and drawer share one `dispatch` that never changes identity, which makes `memo` on cards effective.

CHALLENGE: URL PARSING

Q9. A user edits the URL to `?page=-1&sort=hack&rating=99`. What happens?

`parseFilters` treats the URL as untrusted input: every field has a whitelist or range check and falls back to its default. Sort uses `Object.hasOwn`, not `in`, so `sort=toString` or `sort=constructor` can't match prototype

keys.

Q10. Why leave defaults out of the URL and fix the category order?

Clean URLs (`/` instead of `/?sort=relevance&page=1`), and one filter state maps to exactly one URL. That makes links comparable, cacheable (CDN / React Query keys), and lets us skip writing when nothing changed.

CHALLENGE: DEBOUNCED SEARCH

Q11. Why not just debounce the onChange handler?

The input would become laggy or uncontrolled. The draft must update on every key for a smooth input; only the expensive part (URL write + filtering) is debounced. The draft also has to accept outside changes (Back), which a debounced handler alone doesn't solve.

Q12. Why sync the draft during render instead of in a `useEffect` ?

An effect runs after paint, so for one frame the box shows the old text, and it costs an extra render. Setting state during render when a prop changed is the documented React pattern; React re-renders immediately before painting.

Q13. What if the user types fast and presses Back before the debounce fires?

Back changes `value` , the render-time sync sets the draft to the URL value, and the effect sees `text === value` and schedules nothing; the pending timer is cleared by the effect cleanup. The half-typed text is dropped, which matches what Back means.

CHALLENGE: CART

Q14. The price changed since the user added the item. What do you show?

The new price: the cart only knows the id, the summary joins it with the latest catalog. At checkout the server recalculates anyway; the client total is a preview, never trusted.

Q15. Two tabs are open. The user adds an item in tab A. What does tab B show?

Tab B gets a `storage` event (fired only in *other* tabs) and dispatches `sync` with the parsed value. Writing the same JSON back doesn't fire another event, so there is no ping-pong loop.

Q16. What if localStorage is full, disabled, or holds garbage?

Reads are wrapped in try/catch and sanitised (only positive integers survive). Writes are wrapped too: the cart keeps working in memory for this visit. The key is versioned (`cart:v1`) so a future format can migrate or ignore old data.

IMPLEMENTATION

Q17. Why is `page` reset to 1 on every filter change, but not when you click a page?

New filters mean a new result list; staying on page 4 of a 1-page list shows nothing. `setFilters` spreads `page: 1` before the patch, so a patch that contains `page` (pagination) wins. `paginate` also clamps, covering old links.

Q18. How do you keep cards from re-rendering on every cart change?

`ProductCard` is wrapped in `memo` and receives `qty` (a number) and the stable `dispatch`. Changing one item's qty changes only that card's props. The filter results are memoised on `[products, filters]`, so cart updates don't refilter.

Q19. How does the drawer handle focus and keyboard?

On mount it remembers `document.activeElement`, focuses Close, listens for Escape; on unmount it restores focus to the opener. `role="dialog"`, `aria-modal`, `aria-labelledby`. Next step: a focus trap, or a native `<dialog>` with `showModal()` which gives that for free.

PERFORMANCE / SCALE

Q20. What if the catalog has 1 million products?

Move filtering, sorting, counts and pagination to the server: send `serializeFilters(filters)` as the query string, receive `{ items, total, facets }`. Cancel the previous request with `AbortController` when filters change, cache responses by query string, and keep the previous page visible while loading (`isFetching` instead of a skeleton).

Q21. What if the API is slow?

Skeleton on first load only, then keep showing old results with a subtle loading bar while the new ones come. Debounce search, abort stale requests so a slow old response can't overwrite a newer one, and prefetch the next page.

Q22. Is filtering 48 items on every render a problem?

No, it's microseconds, and it's memoised anyway. `useMemo` matters more for keeping the `filtered` array reference stable than for speed here. With tens of thousands of items, precompute lowercase titles and an index per category.

TRADE-OFFS

Q23. Why not React Router's `useSearchParams`?

In a real app I would use it (or Next's). Here there's a single page, and writing 25 lines with `pushState` + `popstate` shows I understand what the library does. The hook keeps the same `[value, setter]` shape, so swapping it in later touches one file.

Q24. Why pagination and not infinite scroll?

Numbered pages give a shareable, restorable position (`page=3`) and a footer you can reach. Infinite scroll is better for browsing on mobile but needs scroll restoration and a "load more" fallback for accessibility. Asked

in the clarifying questions.

Q25. Why apply price with a Go button instead of live?

Live filtering on a number input commits "1", then "15", then "150" while the user types "1500", re-rendering and pushing history each time, and min / max can swap mid-typing. A form with drafts commits once, supports Enter, and stays predictable. A dual-thumb slider with commit-on-release is the alternative.